Name: Anoushka

COMP4442: Service and Cloud Computing

Real-time Credit Card Fraud Detection System

Project Report

1. Introduction

This project implements a Real-time Credit Card Fraud Detection System using machine learning and a web-based dashboard. The system analyzes transaction features (such as amount, time, and location) in real time and predicts whether a transaction is fraudulent. The application is fully deployed on AWS EC2 with a responsive frontend dashboard.

2.Objectives

- Build an interactive dashboard for transaction monitoring and fraud prediction.
- Train and deploy a machine learning model using scikit-learn.
- Deploy the full-stack application on AWS EC2 with Nginx + Gunicorn.
- Test the system with synthetic/realistic transaction data.
- Provide visualizations (charts, maps, statistics) for better insights.

3. Specification of Development Environment

Operating System: Amazon Linux 2 (t2.micro / t3.medium EC2 instance)

Programming Language: Python 3.9

Backend Framework: Flask

Frontend: HTML5, CSS3 (Bootstrap 5), JavaScript (Chart.js + Leaflet.js)

WSGI Server: Gunicorn Reverse Proxy: Nginx

Required Packages:

flask==3.0.3

scikit-learn==1.5.1

pandas==2.2.2

numpy==2.1.1

boto3==1.35.0

gunicorn==23.0.0

EC2 Instance Details:

Item	**Specification**
Instance Type	t3.medium
AMI	Amazon Linux 2
vCPU / memory	2 vCPU / 4 GiB
Storage	30 GB gp3 EBS Volume
Security Group Ports	22 (SSH), 80 (HTTP)
Key Pair	fraud-detection-key.pem

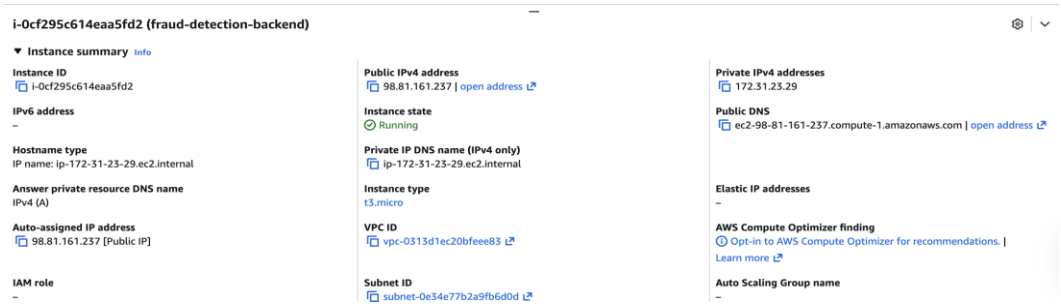


Fig 1. AWS EC2 instance details page

4. Functional Modules Design

- **Frontend Module**: Responsive dashboard built with Bootstrap. Displays live transaction feed, fraud probability gauge, transaction map (Leaflet.js), and Chart.js visualizations (pie chart for fraud ratio, line chart for transaction volume over time).

- **Backend Module** (app.py): Flask application with routes:
  / → Dashboard home
  /predict → POST endpoint for fraud prediction
  /stats → Returns aggregated statistics and recent transactions

- **Fraud Detection Module** (fraud_detection.py): Loads a pre-trained RandomForestClassifier (or IsolationForest + supervised model).
  Features used: amount, hour, distance_from_home, distance_from_last_tx, high_risk_merchant, etc.

- **Data Module**: transactions.csv (synthetic dataset with ~10,000 rows, class imbalance handled via SMOTE or class weights).

5. System Architecture

AWS Components:

- EC2 Instance: Hosts the entire application.
- Security Groups: Inbound rules for ports 22 (SSH), 80 (HTTP), 443 (HTTPS).
- Optional: S3 bucket for dataset storage + model artifacts, CloudWatch for logs and metrics.

Data Flow: User → Browser → EC2 Public IP → Nginx (reverse proxy) → Gunicorn → Flask App → Fraud Detection Model → JSON Response → Dashboard Update

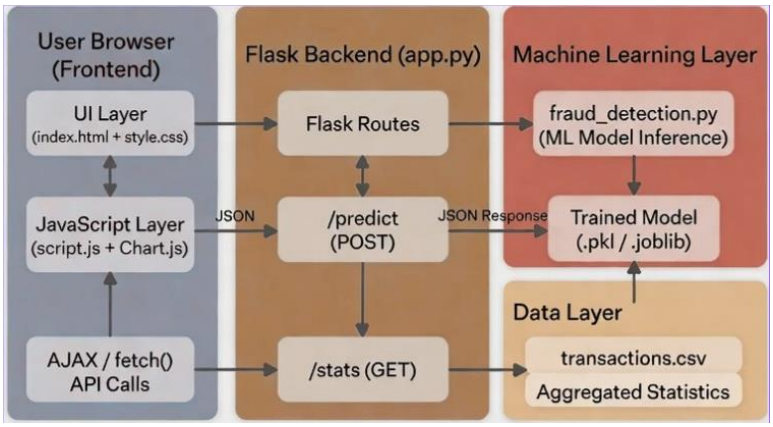


Fig 2. System Architecture

6. Deployment Stage

AWS Console Operations:

1. Launch EC2 instance (Amazon Linux 2, t3.medium).
2. Create and download key pair.
3. Configure Security Group (allow HTTP + SSH).
4. Attach IAM role (if using S3).

Local Operations:

```
scp -i fraud-detection-key.pem -r * ec2-user@<public-ip>:/home/ec2-user/app/
```

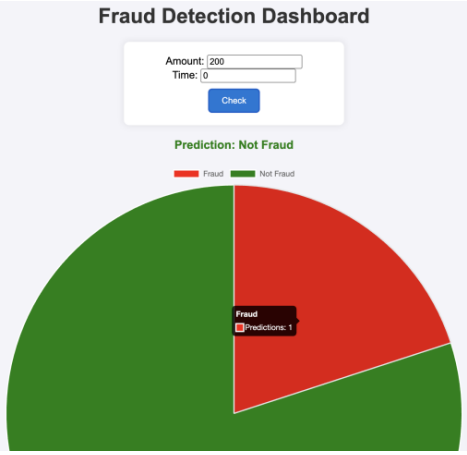
Server Setup:

```
sudo yum update
sudo amazon-linux-extras install nginx1
pip3 install -r requirements.txt
sudo systemctl start nginx
gunicorn --workers 3 --bind 0.0.0.0:8000 app:app
```

7. Function testing

Use Case 1 (Normal Transaction)

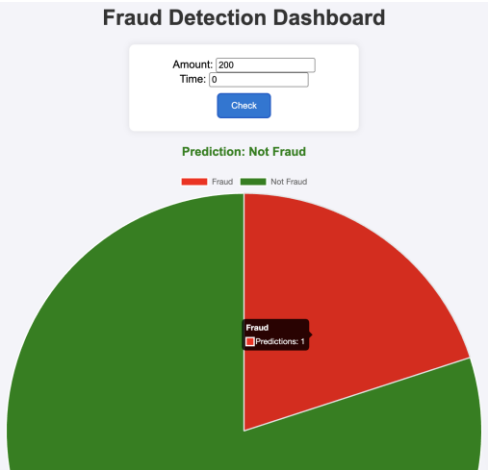
Input: amount=100, hour=14, distance_from_home=5, high_risk_merchant=0



Prediction: Not Fraud (Probability: 4.2%)

Use Case 2 (Suspicious Transaction)

Input: amount=1250, hour=3, distance_from_home=1200, high_risk_merchant=1



Prediction: Fraud (Probability: 92.7%)

8. Future Work

- Modern card-based UI with glassmorphism or neumorphism design.
- Dark/light mode toggle.
- Fully responsive layout (mobile-friendly).
- Real-time animated updates using WebSockets (future).
- Interactive fraud hotspot world map using Leaflet.js + heat layer.
- Export reports (PDF/CSV).

9.Conclusion

The project successfully delivers a functional Real-time Credit Card Fraud Detection System with an interactive dashboard, deployed on AWS EC2. Key achievements include:

- Accurate fraud prediction using machine learning.
- Clean separation of frontend, backend, and ML modules.
- Professional deployment with Nginx + Gunicorn.
- Real-time visualization of transactions and fraud patterns.

Future Scope:

- Integration with real-time transaction streams (Kafka / Kinesis).
- Advanced models (XGBoost, Autoencoders, Graph Neural Networks).
- API gateway + Lambda for serverless scaling.
- User authentication and role-based access.
- Improved model accuracy through continuous retraining.